Week 5 Baseline Models and Evaluation Framework

Objective

This stage uses the Week 4 model-ready Telco matrix to compare simple churn prediction models and methods for handling class imbalance. The input has already been cleaned, engineered, encoded, standardized, and filtered for highly correlated columns.

The main tasks were to create training, validation, and test partitions; compare imbalance strategies; and evaluate Logistic Regression and Decision Tree models using several complementary metrics.

Tenure-based lifecycle split

The dataset has no calendar dates. The historical workflow sorts customers by tenure from high to low and splits the rows as follows:

Partition	Share	Position after sorting
Training	60%	First 60%, generally longer-tenure customers.
Validation	20%	Middle 20%.
Test	20%	Last 20%, generally shorter-tenure customers.

This is a lifecycle-based distribution-shift experiment, not a true time-based forecast. Tenure measures relationship length and does not tell us which customer record occurred earlier in calendar time. The partitions also differ substantially in churn prevalence. A date-based evaluation would require actual observation dates and a defined prediction horizon.

Class imbalance strategies

Strategy	How it works	Purpose
No Resampling	Uses the original training data.	Reference comparison.
Balanced class weights	Assigns greater training weight to the minority class.	Makes missed churn cases more costly during fitting.
RandomOverSampler	Duplicates minority-class training observations.	Increases minority representation.
SMOTE	Creates synthetic minority examples using neighboring observations.	Tests synthetic oversampling.
ADASYN	Creates more synthetic examples near difficult minority observations.	Tests adaptive oversampling.

Resampling is applied only to the training partition. Validation and test rows are not resampled. Standard SMOTE and ADASYN can interpolate encoded categorical values; that is another limitation to consider with this feature matrix.

Baseline model settings

Logistic Regression is an interpretable linear classification baseline. The Decision Tree captures nonlinear splits and simple interaction patterns. In the supplied code, Logistic Regression uses liblinear with max_iter = 1000. The tree uses max_depth = 5 and min_samples_leaf = 30. Model and resampling seeds are set to 42.

Reported validation results

The following values are transcribed from the original Chinese evaluation PDF. They are preserved as historical reported results, not replaced with a new run.

Model	Imbalance strategy	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	Balanced weights	0.5905	0.4526	0.9419	0.6114	0.7882
Decision Tree	None	0.7246	0.6343	0.4606	0.5337	0.7860
Logistic Regression	RandomOverSampler	0.5862	0.4500	0.9440	0.6095	0.7846
Decision Tree	RandomOverSampler	0.6217	0.4723	0.9004	0.6196	0.7840
Logistic Regression	None	0.6572	0.4993	0.6971	0.5818	0.7565
Decision Tree	Balanced weights	0.6828	0.5242	0.7863	0.6290	0.7564
Decision Tree	ADASYN	0.6558	0.4977	0.6598	0.5674	0.7283
Decision Tree	SMOTE	0.6224	0.4684	0.7676	0.5818	0.7219
Logistic Regression	SMOTE	0.5124	0.4057	0.9149	0.5621	0.6739
Logistic Regression	ADASYN	0.5131	0.4061	0.9149	0.5625	0.6620

Balanced Logistic Regression has the highest reported validation ROC-AUC, 0.7882, above the project's baseline target of 0.75. Its recall of 0.9419 means it identifies many churn cases, while its precision of 0.4526 indicates substantial false positives. Random oversampling has slightly higher recall, so balanced Logistic Regression is selected for AUC rather than for the highest recall.

Reported test results

The PDF provides these Logistic Regression test rows:

Imbalance strategy	Accuracy	Precision	Recall	F1	ROC-AUC
None	0.6771	0.6489	0.8677	0.7425	0.7532
Balanced weights	0.6686	0.6329	0.9101	0.7466	0.7501
RandomOverSampler	0.6593	0.6250	0.9127	0.7419	0.7468
SMOTE	0.5394	0.5381	1.0000	0.6997	0.7391
ADASYN	0.5366	0.5366	1.0000	0.6984	0.7345

Test results provide a separate check after validation-based selection. High recall by itself does not imply an effective retention campaign: very broad positive predictions can also produce many false alarms.

The separately supplied English baseline summary reports slightly different scores, including validation AUC 0.7907 and test AUC 0.7479 for balanced Logistic Regression. That version is retained in Report 06. The exact cause of the difference is not established.

Evaluation framework

Metric	Meaning	Use in this project
Accuracy	Share of all predictions that are correct.	Overall reference; can be misleading with unequal classes.
Precision	Share of predicted churn customers who actually churn.	Indicates how efficiently retention resources might be targeted.
Recall	Share of actual churn customers identified.	Indicates how many at-risk customers the model catches.
F1-score	Harmonic mean of precision and recall.	Balances missed churn and false alarms.
ROC-AUC	Ability to rank churn above non-churn across thresholds.	Main validation metric for comparing settings.

Accuracy alone is insufficient because a model can favor the majority class. Recall is important for finding churn cases, precision matters for the cost of contacting customers, and F1 combines those two concerns. The project's AUC target of 0.75 is a stage-specific benchmark, not a universal rule for model quality.

Conclusion and next steps

Balanced Logistic Regression is a useful historical baseline based on validation AUC. More advanced models, including Random Forest and boosting approaches, were proposed for the next stage. Threshold selection could also help align precision and recall with a real retention budget.

The existing matrix was preprocessed before splitting. The results therefore need to be read as prototype results with preprocessing leakage and lifecycle-distribution limitations, rather than as a production performance estimate. A stronger next evaluation would split first, fit preprocessing on training rows only, and reserve the final test set for one final assessment.